

Modern Application 2 Project Report

1. Student Details

Name: Dhanu Sree Suresh

Roll Number: 24F2002559

Email: 24f2002559@ds.study.iitm.ac.in

About Me: I am a highly motivated and detail-oriented AI and Data Science student with a strong foundation in artificial intelligence, machine learning, and data analysis. Proven track record of leading innovative projects and research initiatives, with a passion for leveraging technology to solve real-world challenges. Eager to contribute to impactful industry initiatives by creating compelling presentations, collaborating with experts, and strengthening industry presence.

2. Project Details

Project Title: Placement Portal Application

2.1. Problem Statement:

This Placement Portal Application is a comprehensive web application built with Flask, Vue.js and SQLite. It provides a complete solution for managing placement operations including student registration, student drive applications, drive creation and overall management by the admin

2.2. Approach:

The project is built as a full-stack single page application (SPA) with Flask serving the REST APIs on the backend and Vue.js handling the frontend. JWT authentication is used to secure all API endpoints and role-based authorization. Celery and Redis handles background tasks such as email notifications and report generation. Redis caching is used to optimize frequently accessed data. Bootstrap is used for responsive UI design with Chart.js for interactive visualizations.

2.2.1. Key Features:

- Multi-role Authentication System (Admin, Company, Student)
- Placement Drive Management with eligibility filtering
- Student Application Tracking with status updates
- ATS (Applicant Tracking System) with skill matching
- Company & Interview Reviews with category-based ratings

- Real-time Dashboard with analytics and charts
- Background Job Processing for emails and reports
- Resume Upload & View with role-based access
- Responsive Single-Page Application using Vue.js
- PWA Support for installable web application
- CSV Exports for application history and reports
- Email Notifications for status updates and reminders

2.2.2. Wireframe Models:

Login and Register

Placement Portal

Login

Email

Password

Login

New student? Register.

Company? Register.

Student Registration

Full Name

Email

Password

Roll Number

CoPA

Graduation Year

Register

Existing account? Login

Company Registration

Company Name

Email

Password

Industry

Location

Description

Register

Existing account? Login

Admin Portal

Placement Portal

Company **Drives** **Students**

Name	Email	Approved	Status	Actions
Company 1	hr1@portal.com	No	Active	View Details Edit Deactivate Delete
Company 2	hr2@portal.com	No	Active	View Details Approve Deactivate Delete

Company Details

Name: Company 1
Email: hr1@portal.com
Industry: IT
Location: Address
Description: desc
Drives

Title	Status	Deadline	Actions
Intern 1	Closed	2026-09-25	View Applications

Applicants for Intern 1

Title Name	Roll No.	Branch	CoPA	Status
Student 1	10001	ES	8.5	Selected
Student 2	10002	EE	7.8	Rejected
Student 3	10003	ME	7.2	

Edit Company

FORMS

Save Cancel

Companies **Drives** **Students**

Title	Company	Status	Deadline	Actions
Intern 1	Company 1	closed	2026-09-25	View Details
Intern 2	Company 2	Approved	2027-03-24	View Details

Drive Details

Title: Intern 1
Company: Company 1
Status: closed
Deadline: 2026-09-25
Description: desc
Branch: ...
CoPA Min: ...
Year: ...
Required Skills: ...
Preferred Skills: ...
Applicants:

Name	Roll No.	Branch	CoPA	Status
------	----------	--------	------	--------

Companies **Drives** **Students**

Name	Roll No.	Email	Status	Actions
Student 1	10001	s1@portal.com	Active	Deactivate Delete
Student 2	10002	s2@portal.com	Active	Deactivate Delete

Student Portal

Placement Portal

Available Drives

Role	Company	Details	Deadline	Action
Intern 1	Company 1	desc	2026-09-25	Apply

My Applications

Role	Company	Status	Date
Role 1	Company 2	Selected	2026-02-05

Placement History

Company	Role	Package	Placement Date
Company 2	Role 1	16 LPA	2026-02-05

Company Portal

Placement Portal

Welcome Company 1,

Your Placement Drives

Active

Title	Deadline	Applicants	Actions
Intern 1	2026-09-25	2	View Applicants Edit Close Drive Delete

Closed

No closed drives.

Placement Portal

Applicants for Intern 1

Name	Roll No.	Branch	CGPA	Actions
Student1	10001	CS	8.5	Select Reject Shortlist

Close

3. AI/LLM Declaration

AI was used as a supplementary development tool to assist in code debugging and feedback for improvements. AI was also used as a learning aid to improve the understanding of concepts that were initially unclear, explore different approaches and

gain a better understanding of relevant technologies and best practices. The use of AI was limited to the scope permitted under the AI use policy of the project and all outputs were reviewed, verified and implemented manually in compliance with the requirements of the assignment.

Sample Prompt:

Role

You are a **Senior Full-Stack Web Developer**, **Software Architect**, and **Code Reviewer**. Your task is to perform a comprehensive review of the provided project, evaluating it against software engineering best practices, maintainability, scalability, security, and the technology constraints listed below.

Technology Constraints (Strict)

The application must only use the following technologies unless explicitly stated otherwise.

Backend

- Flask (Application + REST APIs)
- SQLAlchemy ORM
- SQLite (Only database permitted)
- Redis (Caching)
- Celery + Redis (Background/Batch jobs)

Frontend

- Vue.js (Primary UI framework)
- Vue Router
- Vuex (if state management is necessary)
- Axios or Fetch API for API communication

- Bootstrap for styling and layout
- Vanilla JavaScript where appropriate

Templates

- Jinja2 should only be used as the application entry point (when using CDN builds of Vue).
- Jinja2 should ****NOT**** be used for rendering application UI components.

Not Allowed

- Any CSS framework other than Bootstrap
- Any database other than SQLite
- Any frontend framework other than Vue
- Replacing APIs with server-side rendered forms

Evaluation Criteria

Evaluate the project against the following areas.

1. Flask Backend

Review:

- Project structure
- Application architecture
- Routing
- Blueprint organization
- API design
- Error handling
- Request validation
- Configuration management
- Logging

- Separation of concerns

2. Database (SQLite + SQLAlchemy)

Review:

- Database models
- Relationships
- Naming conventions
- Constraints
- Migrations
- Query efficiency
- Index usage
- Data normalization
- Transaction handling

3. Frontend (Vue.js)

Review:

- Component structure
- Vue Router organization
- State management
- API integration
- Reactive design
- Component reuse
- Lifecycle usage
- Performance
- Folder organization

4. Bootstrap UI

Review:

- Layout consistency
- Responsive design
- Accessibility
- Form usability
- Navigation
- User experience
- Bootstrap best practices

5. Authentication & Authorization

Review:

- Login flow
- Logout flow
- Token-based authentication
- Session handling
- Route protection
- Role-based authorization
- Security vulnerabilities

6. CRUD Logic

Review:

- Admin functionality
- User functionality
- API consistency
- Validation
- Error handling
- UI behavior
- Separation between frontend and backend responsibilities

7. Validation

Review both:

Client-side

- Vue validation
- User feedback
- Required fields
- Input formatting

Server-side

- Input validation
- Sanitization
- Error responses
- Business rule validation

8. API Design

Review:

- REST principles

- Endpoint naming
- Status codes
- JSON consistency
- Pagination
- Filtering
- Versioning
- Error responses

9. Charts & Visualization

Review:

- Data presentation
- Chart selection
- Performance
- Readability
- Dashboard usability

10. Background Jobs

Review:

- Celery task organization
- Redis usage
- Retry strategy
- Scheduling
- Failure handling

11. Testing & Debugging

Review:

- Manual testing coverage
- Debugging support
- Logging
- Error reporting
- Development workflow

12. External Integrations

Review:

- API usage
- Error handling
- Timeouts
- Retry logic
- Security
- Configuration

Code Quality Review

Evaluate the codebase for:

- Readability
- Maintainability
- Simplicity
- Modularity
- Reusability

- Separation of concerns
- DRY principle
- SOLID principles
- KISS principle
- Clean Code practices
- Python best practices
- Vue best practices

Security Review

Identify:

- SQL Injection risks
- XSS vulnerabilities
- CSRF protection
- Authentication weaknesses
- Authorization flaws
- Sensitive data exposure
- Hardcoded secrets
- Input sanitization issues
- File upload vulnerabilities
- Rate limiting concerns

Performance Review

Evaluate:

- Database query efficiency
- N+1 query problems

- API performance
- Redis caching opportunities
- Frontend rendering performance
- Bundle size
- Lazy loading opportunities
- Pagination
- Asset optimization

Documentation Review

Evaluate:

README

Review whether it includes:

- Project overview
- Installation instructions
- Dependencies
- Configuration
- Running locally
- Environment variables
- Database setup
- API documentation
- Folder structure
- Deployment instructions

Code Documentation

Review:

- Comments

- Docstrings
- Function documentation
- Module documentation

Architecture Review

Evaluate:

- Overall architecture
- Scalability
- Maintainability
- Extensibility
- Layer separation
- Dependency management

File Structure

The following files will be provided:

```
```text
```

app.py

index.html

README.md

```
```
```

Deliverables

Provide your review in the following format.

1. Executive Summary

A concise overview of the project's strengths and weaknesses.

2. Overall Score

| Category | Score (/10) |
|----------|-------------|
|----------|-------------|

| | |
|-------|-------|
| ----- | ----- |
|-------|-------|

| | |
|--------------|--|
| Architecture | |
|--------------|--|

| | |
|---------|--|
| Backend | |
|---------|--|

| | |
|----------|--|
| Frontend | |
|----------|--|

| | |
|----------|--|
| Database | |
|----------|--|

| | |
|----------------|--|
| Authentication | |
|----------------|--|

| | |
|----------|--|
| Security | |
|----------|--|

| | |
|-------------|--|
| Performance | |
|-------------|--|

| | |
|--------------|--|
| Code Quality | |
|--------------|--|

| | |
|---------------|--|
| Documentation | |
|---------------|--|

| | |
|-----------------|--|
| Maintainability | |
|-----------------|--|

| | |
|---------|--|
| Overall | |
|---------|--|

3. Strengths

List everything implemented well.

4. Weaknesses

List all issues found.

5. Detailed Review

For each issue include:

- File
- Line (if applicable)
- Severity
 - Critical
 - High
 - Medium
 - Low
- Problem
- Why it matters
- Recommended solution

6. Code Quality Review

Discuss:

- Readability
- Function size
- Complexity
- Duplication
- Maintainability

7. Documentation Review

Review both the README and inline documentation.

8. Security Findings

Rank findings by severity and explain how to mitigate each issue.

9. Performance Findings

Highlight bottlenecks and suggest optimizations.

10. Refactoring Suggestions

Recommend architectural and code improvements, prioritizing changes with the highest impact.

11. Best Practices Checklist

Provide a checklist like:

-  Followed
-  Partially followed
-  Not followed

For:

- REST API design
- MVC separation
- Clean Architecture
- SOLID
- DRY
- KISS
- Error handling
- Validation
- Security
- Logging
- Configuration management
- Documentation
- Vue best practices
- Bootstrap best practices
- SQLAlchemy best practices

12. Priority Action Plan

Categorize improvements into:

High Priority

Must be fixed immediately.

Medium Priority

Should be improved soon.

Low Priority

Nice-to-have improvements.

```
---

# Review Guidelines

- Be objective and evidence-based.

- Reference specific code whenever possible.

- Do not make assumptions about missing files.

- Suggest improvements that stay within the permitted technology stack.

- Do not recommend replacing Flask, Vue.js, Bootstrap, SQLite, Redis, or Celery with alternative technologies.

- Prioritize maintainability, security, scalability, readability, and adherence to best practices.

- Where appropriate, provide concise code examples demonstrating recommended improvements.
```

4. Technologies and Frameworks Used

- **Backend:** Flask
- **Database:** SQLite with Flask-SQLAlchemy ORM
- **Frontend:** Vue.js 3, Vue Router, Bootstrap 5, Chart.js
- **Authentication:** Flask-JWT-Extended
- **Background Jobs:** Celery with Redis broker/backend
- **Caching:** Redis using Flask-Caching
- **API Communication:** Axios
- **Security:** Werkzeug for password hashing and validation
- **Templating:** Jinja2
- **Email:** SMTP

| Technology | Purpose |
|------------------|--|
| Flask | Core backend web framework for routing, request handling, and API endpoints |
| Flask-SQLAlchemy | Object Relational Mapper for SQLite database operations and model management |

| | |
|---------------------------|--|
| Flask-JWT-Extended | JWT-based authentication and token management for secure API access |
| Flask-Caching | Redis-backed caching for improved API performance |
| Celery | Distributed task queue for background jobs (email notifications, reports, CSV exports) |
| Redis | Message broker for Celery and caching backend for Flask-Caching |
| SQLite | Lightweight file-based database for local data persistence |
| Vue.js 3 | Reactive frontend framework for building interactive single-page application |
| Vue Router | Client-side routing for navigation between views |
| Bootstrap 5 | Responsive UI framework for consistent styling and layout |
| Chart.js | Interactive chart visualization for analytics and dashboards |
| Axios | HTTP client for API communication between frontend and backend |
| Werkzeug | Password hashing and security utilities |
| SMTP (MailHog) | Email sending for notifications, reminders, and report delivery |

5. Database Schema/ER Diagram

5.1. User Table

| Column | Type | Constraints | Description |
|-----------------------|--------------|----------------------------|-----------------------------------|
| u_id | INTEGER | PRIMARY KEY, AUTOINCREMENT | Unique user identifier |
| email | VARCHAR(120) | UNIQUE, NOT NULL | Login email address |
| password | VARCHAR(256) | NOT NULL | Hashed password |
| role | VARCHAR(20) | NOT NULL | User role (admin/company/student) |
| is_active | BOOLEAN | DEFAULT 1 | Account activation status |
| is_blacklisted | BOOLEAN | DEFAULT 0 | Account blacklist flag |

| | | | |
|----------------|----------|------------------------------|-----------------------|
| created | DATETIME | DEFAULT
CURRENT_TIMESTAMP | Account creation time |
|----------------|----------|------------------------------|-----------------------|

5.2. Company Table

| Column | Type | Constraints | Description |
|--------------------|--------------|----------------------------------|---------------------------|
| c_id | INTEGER | PRIMARY KEY,
AUTOINCREMENT | Unique company identifier |
| user_id | INTEGER | FOREIGN KEY, UNIQUE,
NOT NULL | Reference to User |
| name | VARCHAR(200) | NOT NULL | Company name |
| industry | VARCHAR(200) | | Industry sector |
| location | VARCHAR(500) | | Office location |
| description | TEXT | | Company description |
| website | VARCHAR(500) | | Company website |
| hr_contact | VARCHAR(200) | | HR contact details |
| approved | BOOLEAN | DEFAULT 0 | Admin approval status |

5.3. Student Table

| Column | Type | Constraints | Description |
|--------------------|--------------|----------------------------------|--------------------------------|
| s_id | INTEGER | PRIMARY KEY,
AUTOINCREMENT | Unique student identifier |
| user_id | INTEGER | FOREIGN KEY, UNIQUE,
NOT NULL | Reference to User |
| name | VARCHAR(200) | NOT NULL | Student full name |
| roll_number | VARCHAR(50) | UNIQUE | University roll number |
| branch | VARCHAR(200) | | Academic branch/department |
| cgpa | FLOAT | | Cumulative Grade Point Average |
| year | INTEGER | | Graduation year |
| resume_path | VARCHAR(500) | | Path to uploaded resume |
| skills | VARCHAR(500) | | Skills (comma-separated) |

5.4. Placement Drive Table

| Column | Type | Constraints | Description |
|-------------------------|--------------|----------------------------|---------------------------|
| p_id | INTEGER | PRIMARY KEY, AUTOINCREMENT | Unique drive identifier |
| company_id | INTEGER | FOREIGN KEY, NOT NULL | Reference to Company |
| title | VARCHAR(150) | NOT NULL | Job position title |
| description | TEXT | | Job description |
| branch | VARCHAR(200) | | Eligible branches |
| cgpa_min | FLOAT | | Minimum CGPA requirement |
| year | VARCHAR(200) | | Eligible graduation years |
| deadline | DATETIME | | Application deadline |
| status | VARCHAR(20) | DEFAULT 'pending' | Drive status |
| required_skills | TEXT | | Mandatory skills |
| preferred_skills | TEXT | | Preferred skills |
| technologies | VARCHAR(500) | | Required technologies |
| experience | VARCHAR(100) | | Experience required |
| closed_by_admin | BOOLEAN | DEFAULT 0 | Admin closure flag |

5.5. Application Table

| Column | Type | Constraints | Description |
|-----------------------|-------------|----------------------------|-------------------------------|
| a_id | INTEGER | PRIMARY KEY, AUTOINCREMENT | Unique application identifier |
| student_id | INTEGER | FOREIGN KEY, NOT NULL | Reference to Student |
| drive_id | INTEGER | FOREIGN KEY, NOT NULL | Reference to PlacementDrive |
| status | VARCHAR(20) | DEFAULT 'applied' | Application status |
| applied_date | DATETIME | DEFAULT CURRENT_TIMESTAMP | Application date |
| package | VARCHAR(50) | | Offered package |
| placement_date | DATETIME | | Placement date |

| | | | |
|-------------------|----------------------------------|--|---------------------------------|
| Constraint | UNIQUE(studen
t_id, drive_id) | | Prevents duplicate applications |
|-------------------|----------------------------------|--|---------------------------------|

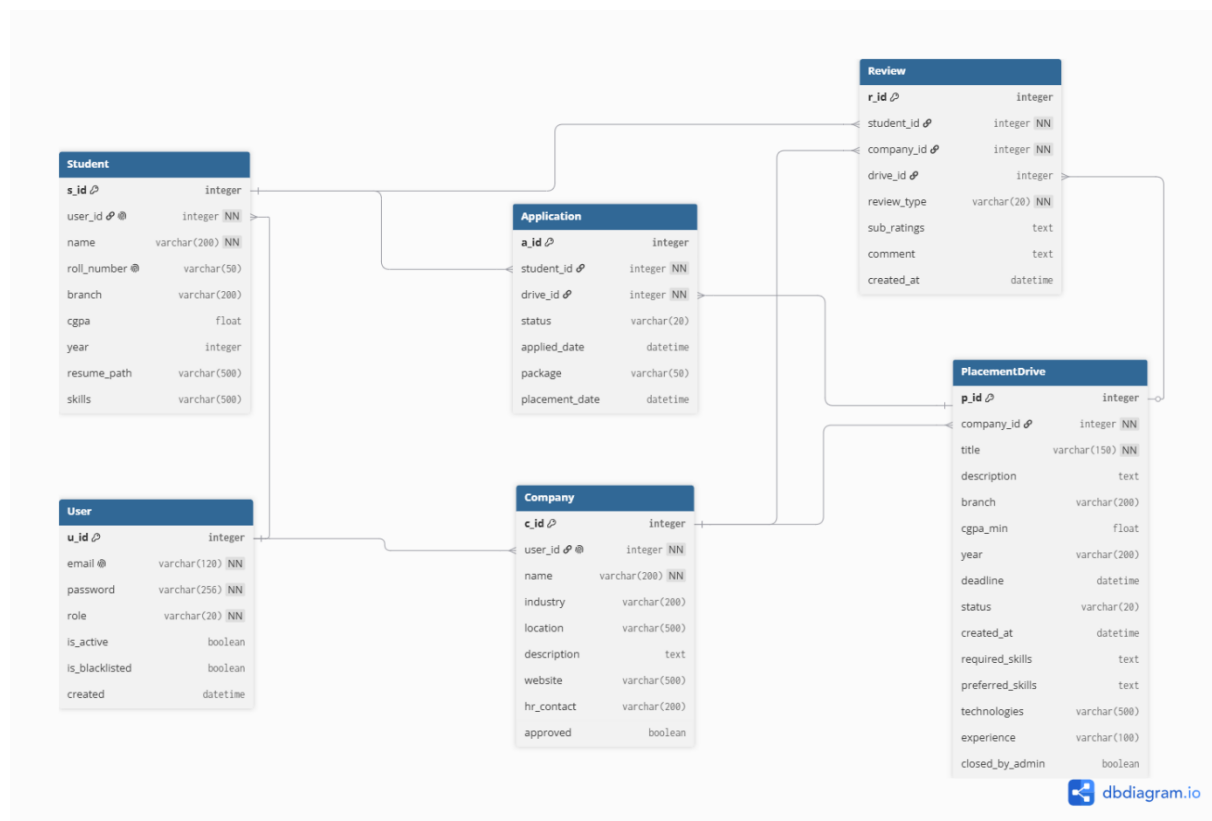
5.6. User Table

| Column | Type | Constraints | Description |
|--------------------|---|-------------------------------|---|
| r_id | INTEGER | PRIMARY KEY,
AUTOINCREMENT | Unique review identifier |
| student_id | INTEGER | FOREIGN KEY, NOT
NULL | Reference to Student (reviewer) |
| company_id | INTEGER | FOREIGN KEY, NOT
NULL | Reference to Company |
| drive_id | INTEGER | FOREIGN KEY | Reference to PlacementDrive (context) |
| review_type | VARCHAR(20) | NOT NULL | company or interview |
| sub_ratings | TEXT | | JSON object with category ratings |
| comment | TEXT | | Text feedback |
| created_at | DATETIME | DEFAULT
CURRENT_TIMESTAMP | Review timestamp |
| Constraint | UNIQUE(studen
t_id,
company_id,
review_type) | | One review per type per student-company |

5.7. Relationships:

- User -> Company: One-to-One (foreign key: user_id)
- User -> Student: One-to-One (foreign key: user_id)
- Company -> PlacementDrive: One-to-Many (foreign key: company_id)
- Company -> Review: One-to-Many (foreign key: company_id)
- Student -> Application: One-to-Many (foreign key: student_id)
- PlacementDrive -> Application: One-to-Many (foreign key: drive_id)
- Student -> Review: One-to-Many (foreign key: student_id)
- PlacementDrive -> Review: One-to-Many (foreign key: drive_id)

5.8. ER Diagram



6. API Resource Endpoints

| Endpoint | Method | Role | Description |
|-----------------------|--------|---------|---|
| /api/login | POST | No Role | Authenticate user and return JWT access token with role |
| /api/register/student | POST | No Role | Register new student account with profile details |
| /api/register/company | POST | No Role | Register new company (pending admin approval) |
| /api/public/stats | GET | No Role | Public statistics (students, companies, drives, placements) |
| /api/admin/dashboard | GET | Admin | Dashboard statistics (totals, pending counts) |
| /api/admin/companies | GET | Admin | List companies with filters (approved/unapproved/active/inactive/blacklisted) |

| | | | |
|--|----------------------|---------|--|
| /api/admin/companies/{id} | GET
PUT
DELETE | Admin | Get, update, or delete company details |
| /api/admin/companies/{id}/drives | GET | Admin | Get all drives for a company |
| /api/admin/companies/{id}/approve | PUT | Admin | Approve company registration |
| /api/admin/companies/{id}/reject | DELETE | Admin | Reject company registration |
| /api/admin/companies/{id}/toggle-active | PUT | Admin | Toggle company active/deactivated status |
| /api/admin/companies/{id}/toggle-blacklist | PUT | Admin | Toggle company blacklist status |
| /api/admin/drives | GET | Admin | List drives with filters (pending/approved/closed) and search |
| /api/admin/drives/{id}/approve | PUT | Admin | Approve placement drive |
| /api/admin/drives/{id}/close | PUT | Admin | Close drive (prevents company reopening) |
| /api/admin/drives/{id} | GET | Admin | Get drive details |
| /api/admin/drives/{id}/applications | GET | Admin | Get applications for a drive |
| /api/admin/students | GET | Admin | List students with filters and search |
| /api/admin/students/{id} | GET
DELETE | Admin | Get student details or delete student |
| /api/admin/students/{id}/applications | GET | Admin | Get student's application history |
| /api/admin/students/{id}/toggle-active | PUT | Admin | Toggle student active status |
| /api/admin/students/{id}/toggle-blacklist | PUT | Admin | Toggle student blacklist status |
| /api/admin/chart-data | GET | Admin | Analytics data for charts (funnel, monthly, company drives, skills) |
| /api/admin/report/{type} | GET | Admin | Download CSV report (placements/companies/drives/applications/stats) |
| /api/company/dashboard | GET | Company | Company dashboard with active/closed drives |
| /api/company/profile | GET | Company | Get or update company profile |

| | | | |
|--|---------------|---------|--|
| | PUT | | |
| /api/company/drives | POST | Company | Create new placement drive (pending approval) |
| /api/company/drives/{id} | PUT
DELETE | Company | Update or delete drive |
| /api/company/drives/{id}/toggle-status | PUT | Company | Toggle drive between approved/closed (unless admin closed) |
| /api/company/drives/{id}/applications | GET | Company | Get applicants with ATS scores (skill matching) |
| /api/company/applications/{id} | PUT | Company | Update application status (shortlist/select/reject) |
| /api/company/ratings | GET | Company | Get aggregated company ratings from reviews |
| /api/company/reviews | GET | Company | Get all reviews for company |
| /api/company/student/{id} | GET | Company | View student profile (resume accessible through token) |
| /api/student/dashboard | GET | Student | Dashboard with eligible drives and application stats |
| /api/student/apply/{drive_id} | POST | Student | Apply for a placement drive (with eligibility checks) |
| /api/student/applications | GET | Student | Get all applications with status |
| /api/student/profile | GET
PUT | Student | Get or update profile (supports multipart for resume upload) |
| /api/student/upload-resume | POST | Student | Upload resume file |
| /api/student/resume/{student_id} | GET | Student | Serve resume file (allowed for student, company, admin) |
| /api/student/history | GET | Student | Placement history with review status |
| /api/student/export-csv | POST | Student | Trigger CSV export through Celery (email delivery) |
| /api/student/export-csv-download | GET | Student | Download CSV directly |
| /api/student/ats-check | POST | Student | ATS skill matching for a drive |
| /api/student/review | POST | Student | Submit/update company or interview review |
| /api/student/reviews | GET | Student | Get all reviews submitted by student |

YAML API Definition File:

Included separately in the submission ZIP as **api.yaml**.

7. Architecture and Features

7.1. Application Architecture

7.1.1. Business Logic

Placement Drive Workflow

- **Company Registration:** Company registers, pending admin approval.
- **Admin Approval:** Admin reviews and approves company.
- **Drive Creation:** Approved company creates placement drive (pending approval).
- **Drive Approval:** Admin approves drive, making it visible to students.
- **Student Application:** Eligible students apply before deadline.
- **Company Review:** Company reviews applications, updates status (shortlist/select/reject).
- **Selected:** Student receives offer letter through email, placement recorded.

Applicant Tracking System

- Extracts required skills from drive (required, preferred, technologies).
- Extracts student skills from profile.
- Computes match percentage based on skill overlap.
- Displays matched and missing skills to company.
- Provides score and explanation for each applicant.

Review System

1. **Company Reviews:** Submitted by selected students.
 - **Categories:** culture, compensation, career, work_life, senior_mgmt, diversity, process, difficulty
 - Aggregated anonymously for company viewing.
2. **Interview Reviews:** Submitted by shortlisted students.
 - **Categories:** process, difficulty
 - Aggregated anonymously for company viewing.
3. One review per student-company-review_type combination.

Email Notifications

1. **Application Status Updates:** Depending on application status, email is sent.
 - **Selected:** Offer letter with company details.
 - **Shortlisted:** Next steps notification.

- **Rejected:** Polite rejection.
- 2. **Daily Reminders:** For drives with upcoming deadlines (8:00 AM GST daily).
- 3. **Monthly Reports:** Summary to admin (1st of each month).
- 4. **CSV Exports:** Application history through email.

7.1.2. Data Integrity

- Students can only apply once per drive.
- Eligibility checks prevent ineligible applications (branch, CGPA, year).
- Admin-closed drives cannot be reopened by company.
- Reviews limited to one per student-company-review_type.

7.1.3. Architecture Description

Backend: Flask application with monolithic structure (single app.py)

- **Models:** SQLAlchemy ORM with relationships and constraints.
- **Routes:** API endpoints grouped by role (admin, company, student).
- **Business Logic:** Helper functions (ratings, ATS, email, validation).
- **Background Tasks:** Celery tasks (daily reminders, monthly reports, CSV exports).
- **Caching:** Redis through Flask-Caching for dashboard data and stats.
- **Authentication:** JWT with Flask-JWT-Extended.

Frontend: Vue.js 3 Single-Page Application (SPA) inside templates/index.html

- **Components:** Role-based views (AdminDashboard, CompanyDashboard, StudentDashboard, PublicDashboard, Login, Register).
- **Routing:** Vue Router for client-side navigation.
- **API Integration:** Axios with automatic JWT injection.
- **Styling:** Bootstrap 5 with custom CSS variables for role-based theming.
- **Charts:** Chart.js for admin analytics.

Data Flow:

- Vue components make Axios calls to Flask endpoints.
- Flask validates JWT, checks role, performs business logic.
- SQLAlchemy executes database queries.
- JSON responses returned to Vue components.
- Vue updates reactive DOM.

Background Processing:

- Celery workers handle async tasks (email sending, CSV generation)
- Redis used as broker and result backend
- Celery Beat schedules daily and monthly jobs

Caching Strategy:

- Admin dashboard and list endpoints cached for 60-120 seconds
- Cache invalidated on data mutations (create/update/delete)

7.2. Implemented Features

1. Authentication System

- JWT-based login with role-specific redirects
- Secure password hashing using Werkzeug
- Student and company registration flows

2. Admin Dashboard

- Statistics cards (students, companies, drives, pending approvals)
- Company management (list, view, edit, approve/reject, activate/deactivate, blacklist)
- Drive management (list, approve, close, view applicants)
- Student management (list, view, activate/deactivate, blacklist, delete)
- Analytics with Chart.js (funnel chart, company drives, monthly trends, skill demand)
- CSV report exports (placements, companies, drives, applications, stats)

3. Company Dashboard

- Profile management (edit company details)
- Placement drive CRUD (create, edit, delete)
- Drive status toggling (open/close)
- Applicant viewing with ATS (Applicant Tracking System) scoring
- Application status updates (shortlist, select, reject)
- Ratings and reviews viewing

4. Student Dashboard

- Profile management (edit company details)
- Placement drive CRUD (create, edit, delete)
- Drive status toggling (open/close)
- Applicant viewing with ATS (Applicant Tracking System) scoring
- Application status updates (shortlist, select, reject)
- Ratings and reviews viewing

7.3. Implemented Features

1. **Resume Upload & View:** Secure file upload with role-based access (student/company/admin can view)
2. **ATS (Applicant Tracking System):** Skill-based matching comparing student skills with drive requirements

3. **Company & Interview Reviews:** Anonymous ratings with category-based sub-ratings (culture, compensation, career, work_life, senior_mgmt, diversity, process, difficulty)
4. **Email Notifications:** Automated emails for application status changes (selected, rejected, shortlisted)
5. **PWA Support:** Web App Manifest and service worker for installable application
6. **Responsive UI:** Bootstrap-based design with role-specific color theming
7. **Interactive Charts:** Chart.js for admin analytics (funnel, pie, line, bar charts)

8. Video Presentation

<https://drive.google.com/file/d/1uKutR23q2BNcreS0Sg1zhU-Hil4dLBDj/view?usp=sharing>

9. References

9.1. Books Referred

1. Flask Web Development 2nd Edition - Miguel Grinberg
2. Learning Web Design 5th Edition - Jennifer Niederst Robbins
3. Database System Concepts 7th Edition - Abraham Silberschatz, Henry F. Korth, S. Sudarshan
4. Eloquent JavaScript: A Modern Introduction to Programming 4th Edition - Marijn Haverbeke
5. You Don't Know JS: Up and Going - Kyle Simpson
6. You Don't Know JS: Types and Grammar - Kyle Simpson
7. You Don't Know JS: ES6 and Beyond - Kyle Simpson
8. You Don't Know JS: Async and Performance - Kyle Simpson
9. You Don't Know JS: this and Object Prototypes - Kyle Simpson
10. You Don't Know JS: Scope and Closures - Kyle Simpson
11. The Jamstack Book: Beyond Static Sites With JavaScript, APIs, and Markup - Raymond Camden, Brian Rinaldi
12. Vue.js 3 for Beginners: Learn the essentials of Vue.js 3 and its ecosystem to build modern web applications - Simone Cuomo

9.2. Online Resources Used

1. <https://colors.co/> (Colour Palette)

2. <https://getbootstrap.com/docs/5.0/getting-started/introduction/> (Bootstrap Documentation)
3. <https://www.w3schools.com/bootstrap5/index.php> (Bootstrap Tutorial)
4. <https://flask.palletsprojects.com/en/stable/> (Flask Documentation)
5. <https://flask-sqlalchemy.readthedocs.io/en/stable/> (Flask-SQLAlchemy Documentation)
6. <https://werkzeug.palletsprojects.com/en/stable/> (Werkzeug Documentation)
7. <https://vuejs.org/guide/introduction> (Vue.js Documentation)
8. <https://docs.celeryq.dev/en/stable/> (Celery Documentation)
9. <https://flask-jwt-extended.readthedocs.io/en/stable/> (Flask-JWT-Extended Documentation)
10. <https://storyset.com/> (Website Illustrations)